COMP202 Data Structures and Algorithms

Assignment 1: Operation Fastest Route

Instructor: Alptekin Küpçü

Due Date: December 19, 2025, 11:59 PM

General Information

This assignment tests your understanding of **graphs**, **priority queues**, and **disjoint-set (union-find) data structures**. You will implement **Dijkstra's shortest–path algorithm** and **Kruskal's minimum spanning tree algorithm** entirely from scratch, without using Java's built-in collections.

This assignment must be completed individually. Discussion of high-level concepts is permitted, but sharing or copying code is strictly forbidden and constitutes plagiarism. The use of AI code generation tools is not permitted. By submitting your work, you agree to abide by the rules in the course syllabus and the Koç University Student Code of Conduct

Academic Integrity Statement

To receive a non-zero grade, you must submit a photo of the provided statement from Honor.pdf. The statement itself can be printed, but the section for your personal details must be filled out by hand using a **blue ink pen**. Acceptable file formats are .pdf, .jpg, or .png. Please do not upload .heic files as they may not be viewable.

Important Note: Please also upload the filled out Honor.pdf in your code submission

Assignment Overview: Operation Fastest Route

Intelligence reports indicate a network of secret communication routers operated by an unknown organization. Your tasks are:

Part 1:

Reconstruct optimal paths between critical routers and determine the shortest travel time between nodes using **Dijkstra's algorithm**.

Part 2:

Determine the optimal way to connect all routers with minimal total connection cost using **Kruskal's minimum spanning tree algorithm**.

To complete the mission, you must implement:

1. a weighted graph,
2. a union-find data structure,
3. Dijkstra's algorithm, and
4. Kruskal's algorithm.

Notes:

- The input may contain both directed and undirected edges.
    - For Dijkestra, undirected edge indicates edges in both directions. Your Graph must convert undirected edges into two directed edges before running Dijkstra.
    - For Kruskal, edges are treated as undirected
- If multiple edges exist between the same pair of vertices, treat them as separate edges, but your algorithms should naturally select the minimum path (Dijkstra) or minimum spanning tree (Kruskal).

Technical Constrains

- **No Built-in Collections**: You are strictly forbidden from importing or using `java.util` collections (`ArrayList`, `HashMap`, `LinkedList`, etc.).
- **Data Structures**: You may reuse helper functions or data structures you implemented in previous projects or create your own helper functions.
- **Provided Structure**: You will be given a `src` folder. You must only fill in the `//TODO` sections in the `ds` package.
    - `Graph.java` // with TODO
    - `Dijkstra.java` // with TODO
    - `UnionFind.java` // with TODO
    - `Kruskal.java` // with TODO
    - `Heap.java` // fully implemented, you may use it or reuse your own from previous projects
    - `Main.java` // used for grading and testing

Grading Breakdown

- Implementation (100 Points): Your grade for the coding portion will be based on the correctness of your Java code, as evaluated by the autograder in Main.java.
    - Correct Dijkstra implementation: 50 points
    - Correct Kruskal implementation: 50 points

Submissions

Upload the following to LearnHub:

1. Graph.java
2. Heap.java
3. Dijkstra.java
4. Kruskal.java
5. UnionFind.java
6. Your handwritten Academic Integrity Statement (as .jpg, .png, or .pdf)

Important:
We provide a Main.java for you to test your code. Passing these local tests is a good start but does not guarantee a full grade. Your final implementation grade will be determined by a more comprehensive, hidden auto grader.